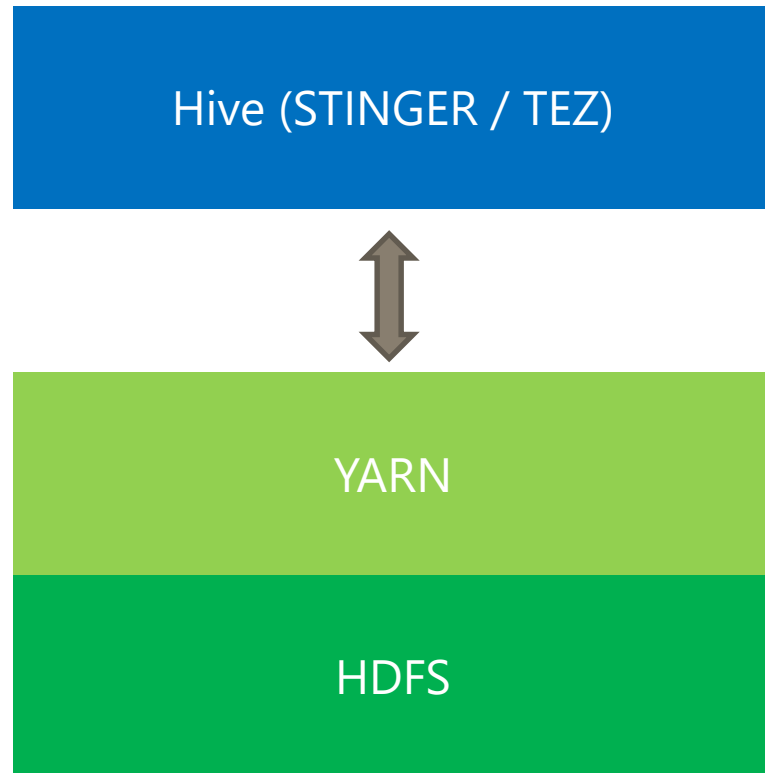
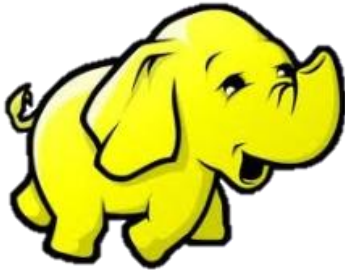

Fundamentos de Hive

Agenda

- **Introducción a Hive**
- Arquitectura Hive
- HiveQL y Almacenamiento
- Particiones y cubos
- Estrategias de Join
- Evolución Hive: Tez y LLAP

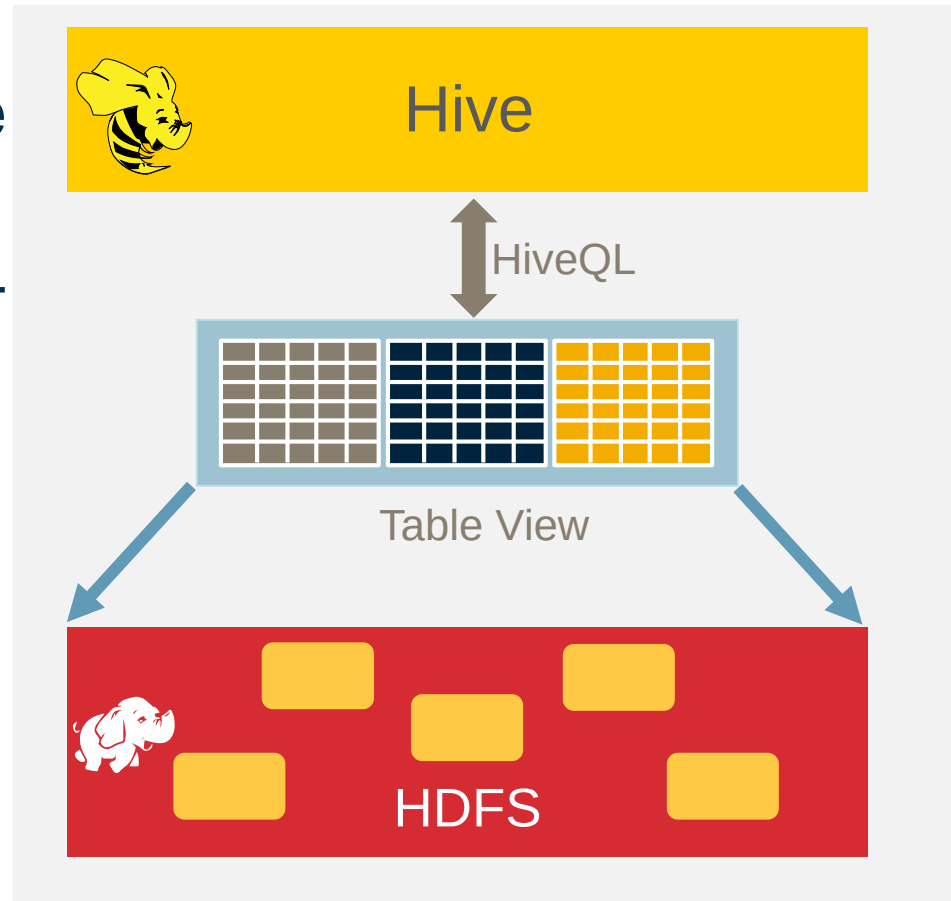
Introducción

- Sistema Data Warehouse para Hadoop



Introducción

- Permite exponer como “tablas” el contenido de HDFS
- Lenguaje similar a SQL llamado HiveQL
- Permite extender el lenguaje HiveQL
- Estructuras de datos con tipos de datos “ricos” (structs, listas y mapas)
- Consultas de datos de diferentes formatos: texto / binario



Introducción

- Facilita sumalizaciones de datos
- Consultas Ad-hoc
- Análisis de grandes cantidades de datos almacenados en Hadoop

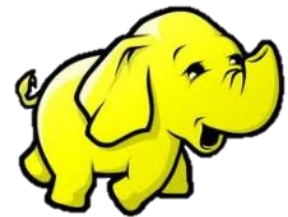
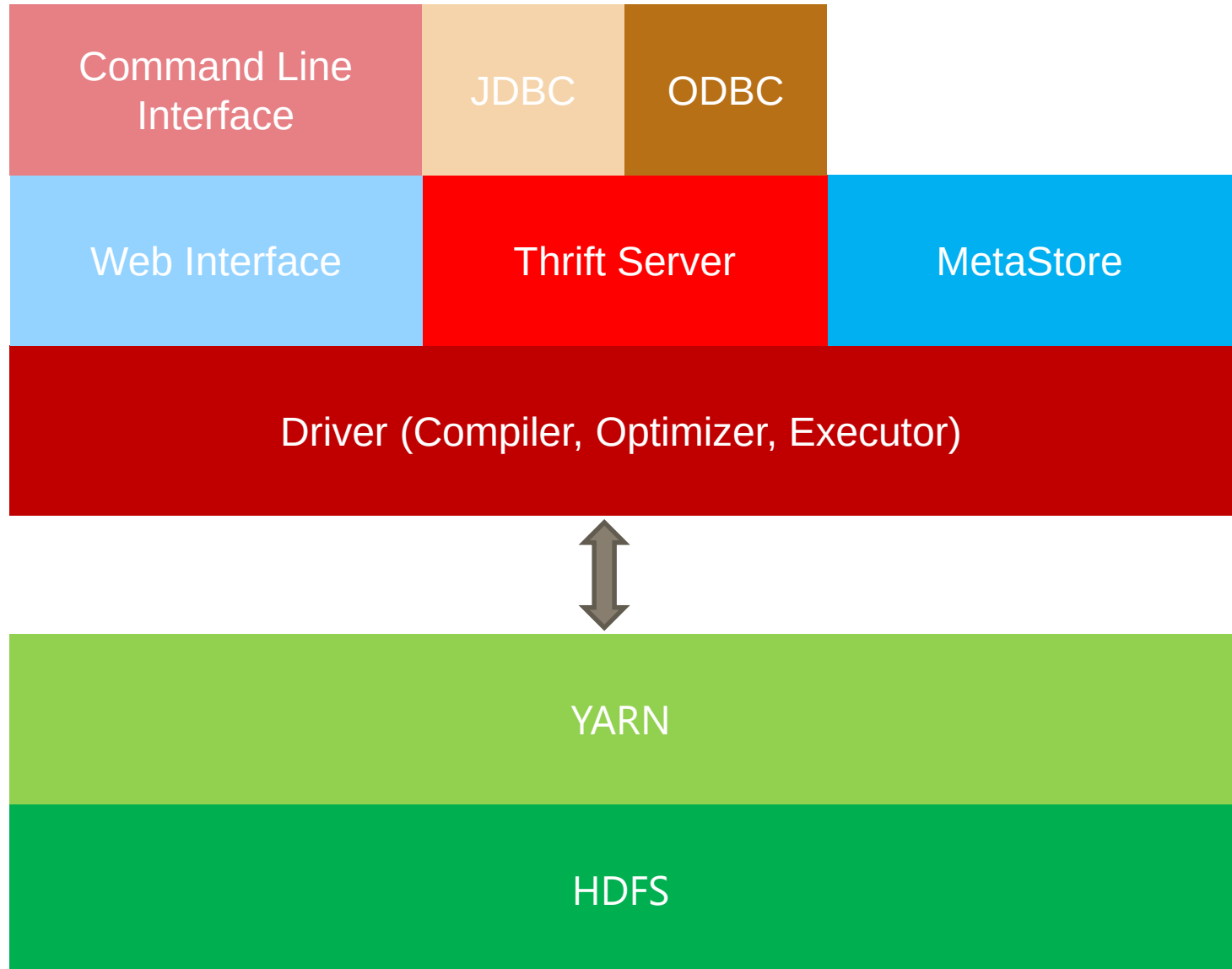
¿Por qué Hive?

- Map Reduce duro de programar
- Carecen de expresividad
- SQL es un lenguaje familiar
- Permite consultar de forma estructurada datos no estructurados
- Bueno para Análisis
- Rendimiento

Agenda

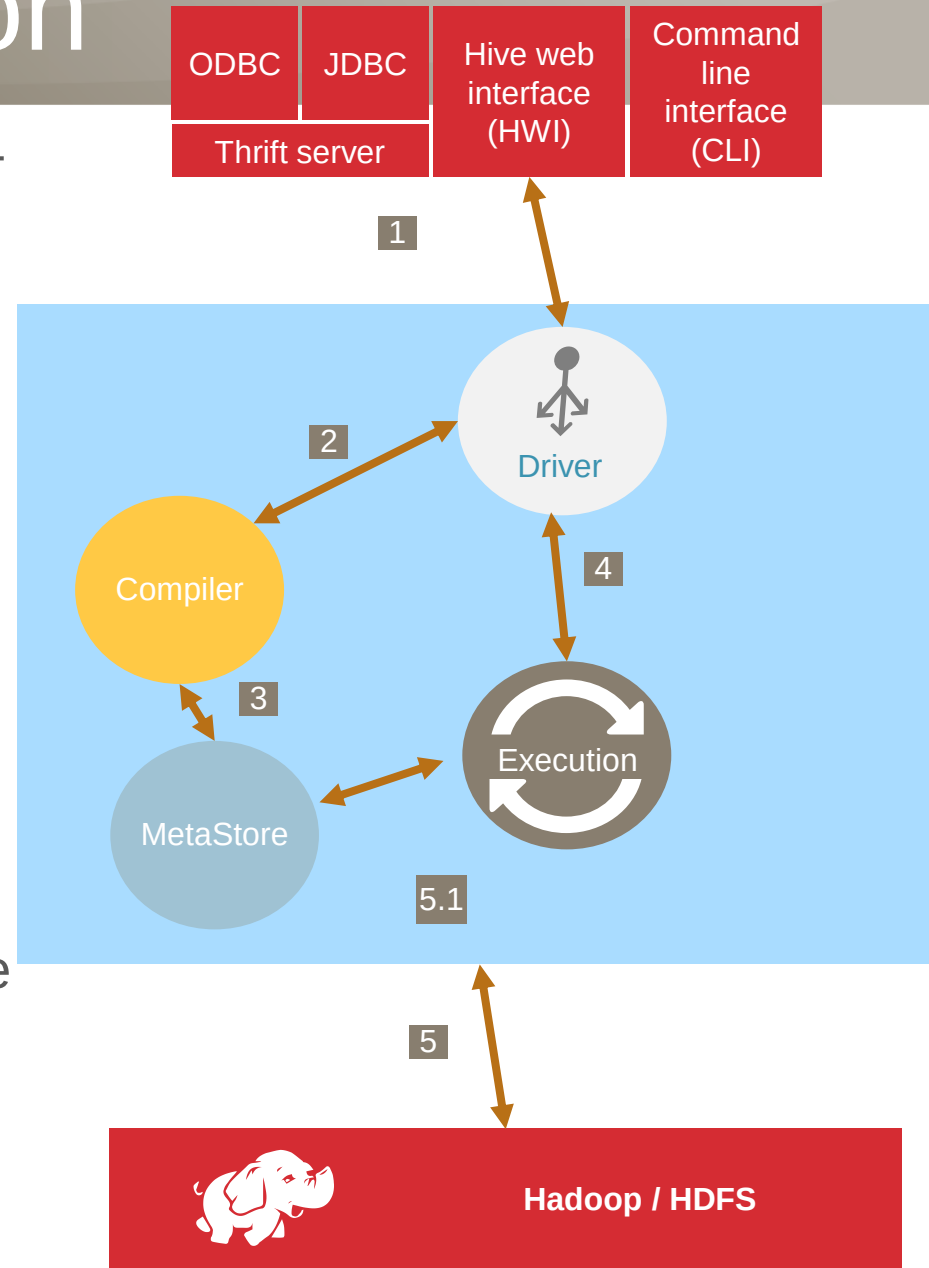
- Introducción a Hive
- **Arquitectura Hive**
- HiveQL y Almacenamiento
- Particiones y cubos
- Estrategias de Join
- Evolución Hive: Tez y LLAP

Arquitectura Hive



Flujo de Ejecución

- 1 Cliente Hive envía consulta HiveQL al driver
- 2 El driver envía la consulta al compilador para comprobación de sintaxis y parsing.
- 3 El compilador obtiene metadatos del almacén y devuelve un plan de ejecución al driver.
- 4 El driver envía el plan de ejecución al motor.
- 5 El motor de ejecución envía el plan como un trabajo Map Reduce. Una vez completada la ejecución el motor obtiene los resultados que se devuelven al driver que los devuelve al cliente Hive.
- 5.1 El motor de ejecución puede ejecutar operaciones de metadatos contra el Metastore



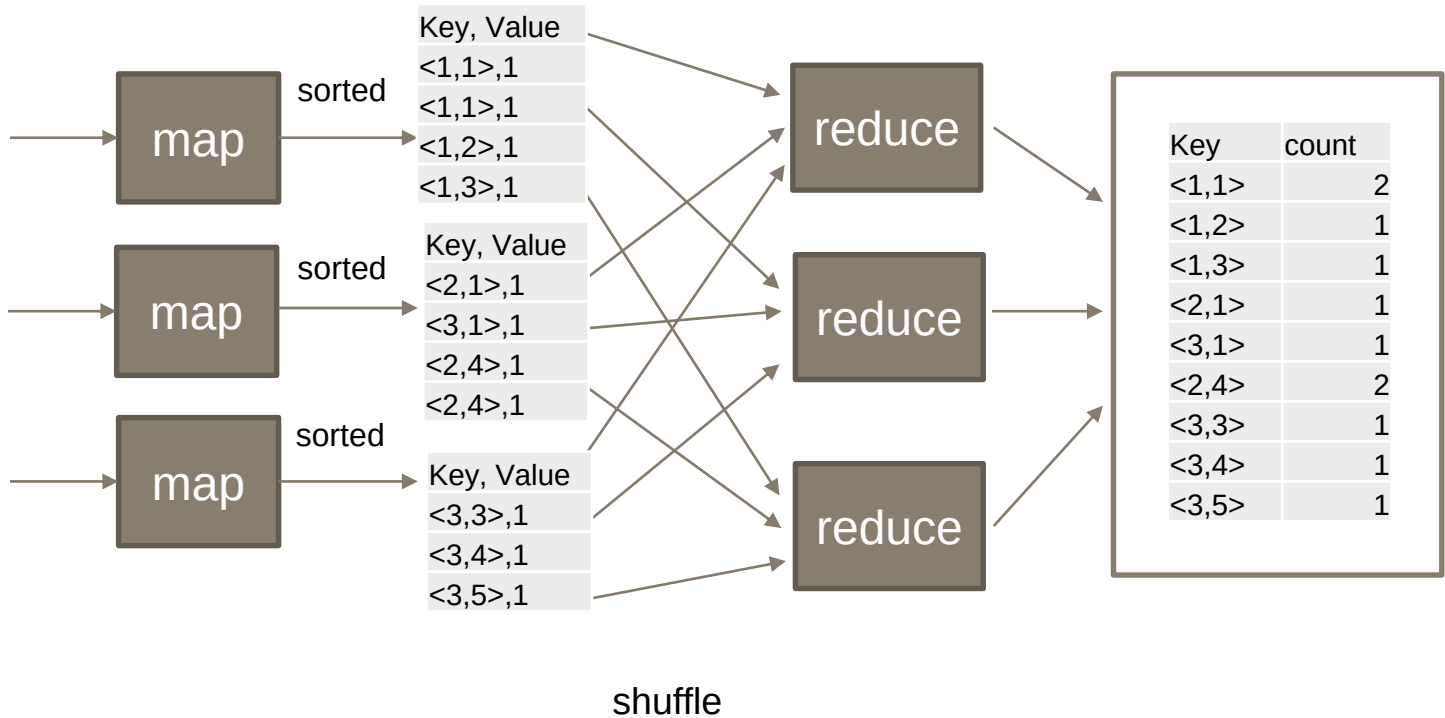
Agrupar por ubicación, userid

► 3 Mappers, 3 Reducers

- Trabajo para una consulta Hive

HIVE TABLE

locationid	userid
1	1
1	2
1	3
1	1
2	4
2	4
2	1
3	1
3	3
3	5
3	4



Demostración



Consultas HIVE simples

Agenda

- Introducción a Hive
- Arquitectura Hive
- **HiveQL y Almacenamiento**
- Particiones y cubos
- Navegando Hive con Excel
- Estrategias de Join
- Evolución Hive: Tez y LLAP

HiveQL vs SQL-92

Caract.	SQL	HiveQL
Updates	UPDATE, INSERT, DELETE	INSERT OVERWRITE, NO DELETE NO UPDATE
Transacciones	Soportado	No Soportado
Índices	Soportado	No Soportado
Latencia	Sub-segundo	Minutos
Funciones	Cientos de funciones	Docenas de funciones
Inserts Multitabla	No Soportado	Soportado
Create table as select	No válido en SQL-92 pero disponible en varios RDBS	Soportado
Select	SQL-92	Solo un objeto en el FROM. SORT BY para ordenación parcial. LIMIT para limitar filas
Joins	Soportado	Soportado
Subqueries	Cualquier clausula. Correlacionada o no	Solo cláusula FROM. Subconsultas correlacionadas no soportadas
Vistas	Actualizable. Materializada para la no materializada	Solo lectura. Materialización no soportada
Puntos extensión	Funciones de usuario. Procedimientos almacenados	Funciones de usuario. Scripts MapReduce.

Tipos de Datos

Type	Description
TINYINT	1 byte signed integer
SMALLINT	2 byte signed integer
INT	4 byte signed integer
BIGINT	8 byte signed integer
FLOAT	4 byte single precision floating point number
DOUBLE	8 byte double precision floating point number
BOOLEAN	true/false
STRING	Character string 'a', "a"
ARRAY	An ordered collection of fields, must all be of the same type
MAP	An unordered connection of Key-values pairs. Keys must be primitives. Keys must be the same type, values must be the same type. map('a',1,'b',2)
STRUCT	A collection name fields. The fields may be of different types. struct ('a',1,1.0)

Almacenamiento de Tablas(I)

- Datos almacenados en HDFS
- La operación de carga es una copia de fichero
- La tabla especifica como parsear el fichero para devolver los datos.
- Se denomina esquema o lectura
- No todos los datos de los ficheros tienen que utilizarse en la definición de la tabla
- Rendimiento en la carga (no parse), flexibilidad en consultas

Almacenamiento de tablas(II)

- Tablas Administradas
 - Los ficheros se copian al directorio hive
- Tablas externas
 - Referencias a ficheros en su ubicación original

Almacenamiento de Tablas(III)

- Formato de fila
 - Filas y campos en una fila
 - Por defecto: texto delimitado
- Formato Fichero
 - Formato contenedor
 - Por defecto: fichero de texto
 - Existen también formatos binarios orientados a columnas (RCFiles) y filas

Cómo ejecutar una consulta

- Hive CLI
 - El cliente pesado HIVE
 - \$hive
 - Hive>
- Beeline
 - Nueva consola de comandos que conecta a una instancia HiveServer2
 - \$ beeline -u url -n username -p password
 - beeline>
- Vistas de Ambari / Herramientas como Visual Studio

HiveQL CREATE TABLE

```
CREATE [EXTERNAL] TABLE table_name  
[(col_name data_type [COMMENT col_comment], ...)]  
[ [ROW FORMAT row_format] [STORED AS file_format] ]  
[LOCATION hdfs_path]
```

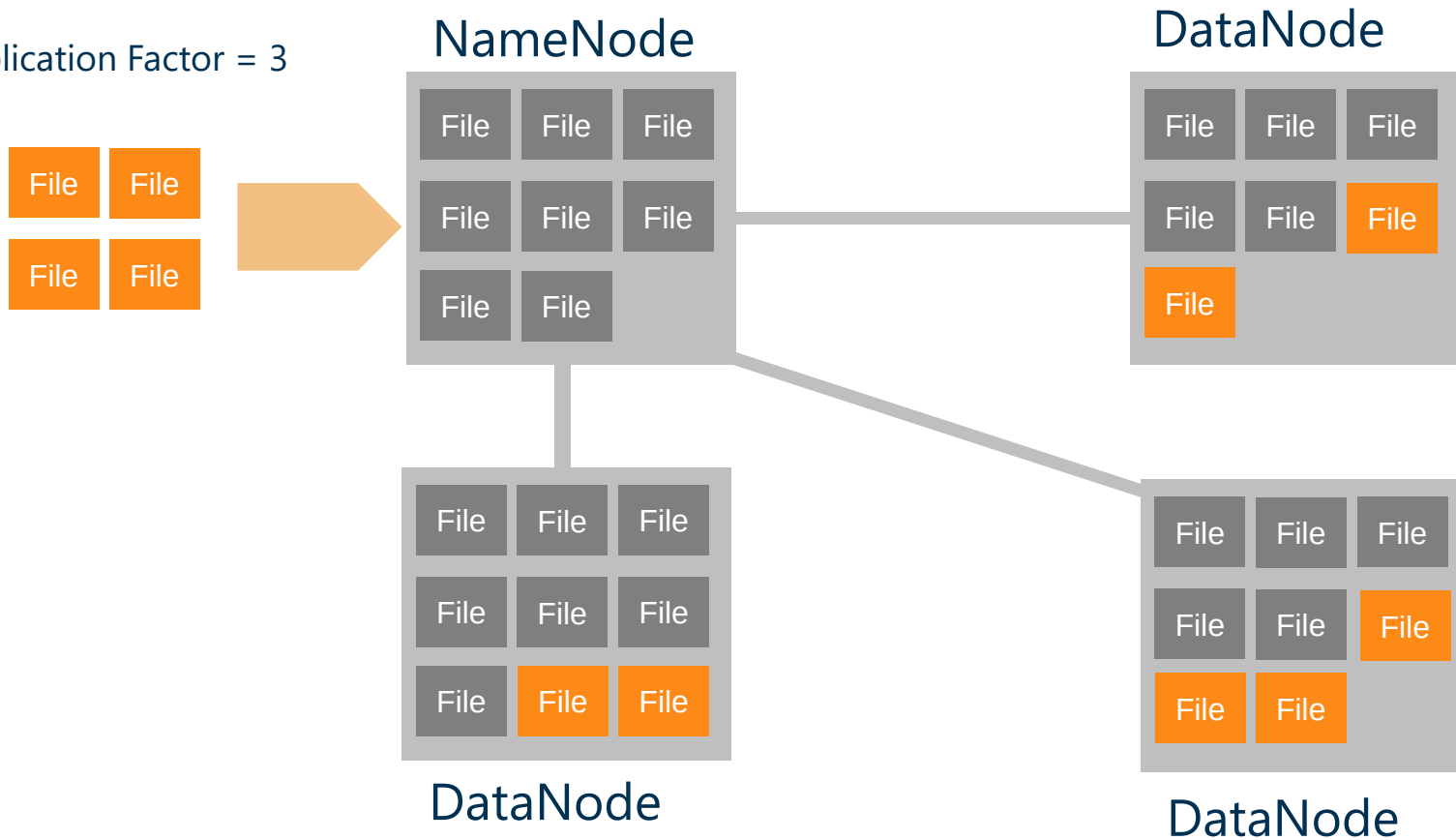
```
CREATE EXTERNAL TABLE log (gid INT, name STRING,  
pubname STRING, releaseDate STRING, rating DOUBLE)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'  
LOCATION '/user/paco/log';
```

- Si no se especifica External, los datos se mueven a la carpeta hive
- Tab como delimitador de campo

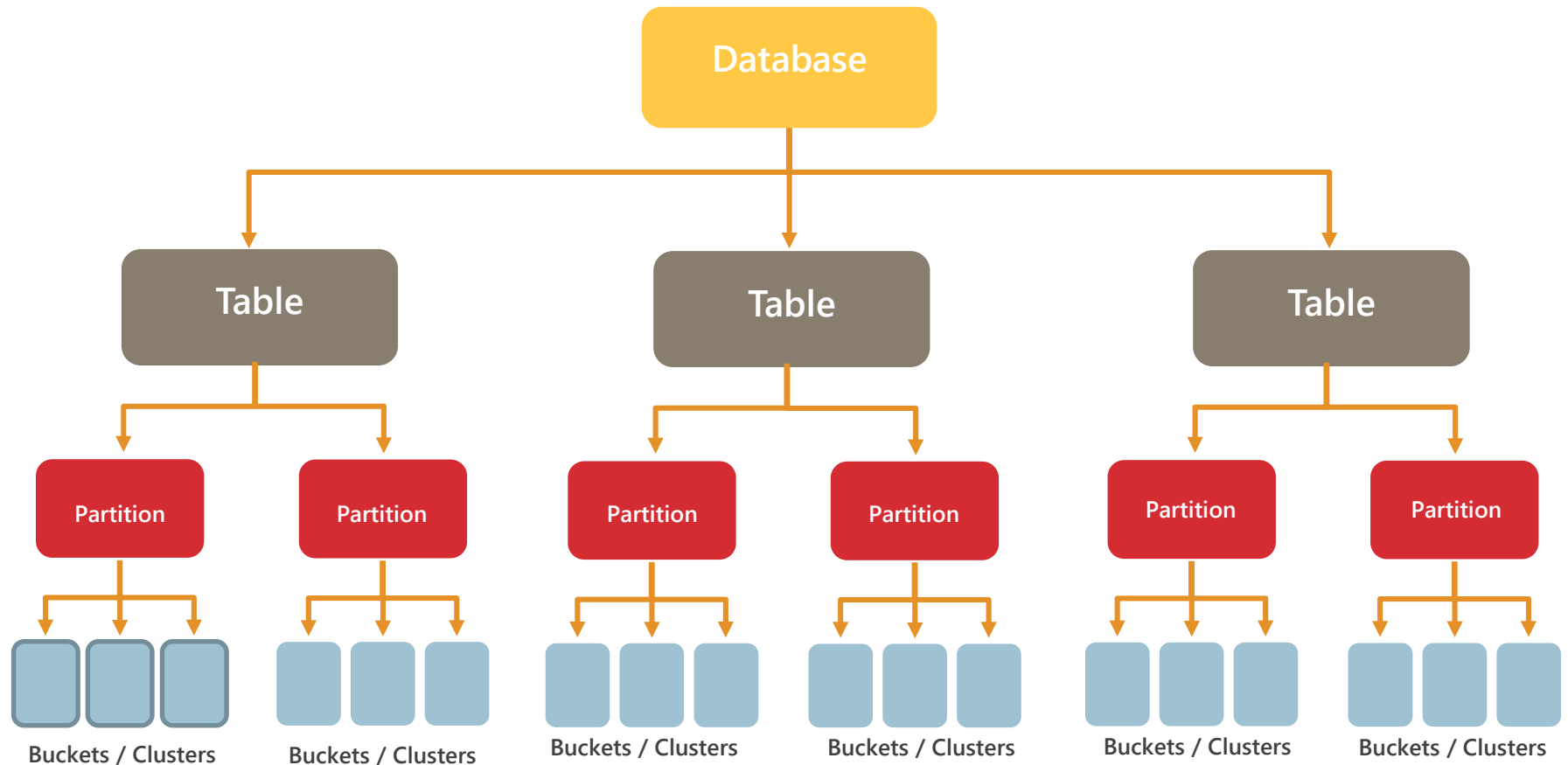
Recordemos HDFS

Block Size = 64 Mb

Replication Factor = 3



Organización Jerárquica datos



Demostración



CREATE TABLE

HiveQL ALTER TABLE

--Rename Table

```
ALTER TABLE tableName RENAME TO newTableName
```

--Change column name, type, and position

```
ALTER TABLE tableName CHANGE col_old_name col_new_name  
column_type [COMMENT comment] [FIRST|AFTER colName]
```

--Add or replace column

```
ALTER TABLE tableName ADD|REPLACE  
COLUMNS (col_name data_type [COMMENT col_comment], ...)
```

HiveQL DROP TABLE

`DROP TABLE tableName`

- Borra los ficheros si la tabla es administrada, si es externa elimina la definición de tabla

HiveQL Otros comandos DDL

CREATE/DROP DATABASE dbName

CREATE/DROP VIEW viewName AS SELECT ...

CREATE/DROP FUNCTION funcName AS className

CREATE/DROP INDEX idxName ON TABLE tblName (colName)

SHOW DATABASES/TABLES/COLUMNS

Creación de una Tabla

Soporta tipos
complejos
como "map"

Internal Tables

Se gestiona / controla los datos. Si una tabla interna se borra, los datos asociados se eliminan. Predeterminada

External Tables

No controla / gestiona los datos. Si se elimina los datos asociados no se borran

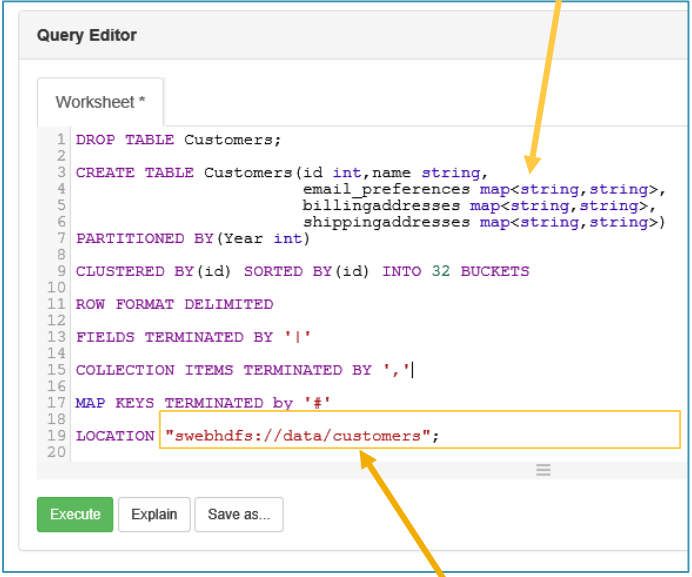
LOCATION

Define la ubicación de los datos

- Puede ser una carpeta HDFS que ya contiene datos. O podemos agregar datos más tarde a una carpeta vacía
- Si no se especifica el predeterminado es: /hive/warehouse/

Varias vistas

Podemos tener varias tablas apuntando a los mismos datos



```
1 DROP TABLE Customers;
2
3 CREATE TABLE Customers(id int,name string,
4                          email_preferences map<string,string>,
5                          billingaddresses map<string,string>,
6                          shippingaddresses map<string,string>)
7 PARTITIONED BY(Year int)
8
9 CLUSTERED BY(id) SORTED BY(id) INTO 32 BUCKETS
10
11 ROW FORMAT DELIMITED
12
13 FIELDS TERMINATED BY '|'
14
15 COLLECTION ITEMS TERMINATED BY ','
16
17 MAP KEYS TERMINATED BY '#'
18
19 LOCATION "swebhdfs://data/customers";
20
```

HDInsight puede apuntar directamente a directorios de ADLS

HiveQL INSERT

--INSERT

```
INSERT INTO tableName  
SELECT ...
```

--LOAD

```
LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE]  
INTO TABLE tablename
```

Carga de Datos

Opciones

Cargar datos locales

```
LOAD DATA LOCAL INPATH ... INTO TABLE ...  
hadoop dfs -put 'local source' 'destination'
```

Cargar datos de otras tablas Hive

```
INSERT INTO TABLE 'destination-hive-table'  
SELECT 'select-statement' FROM 'source-hive-table'
```

Insertar valores directamente

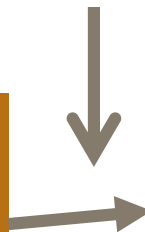
```
INSERT INTO TABLE 'tablename'  
VALUES (row1_values), (row2_values), ....
```

```
LOAD DATA LOCAL INPATH '/home/dataset/customers_address_hive'  
OVERWRITE INTO table Customers PARTITION (year=2015);|
```



Local
File

'/home/dataset/customers_address_hive'



/hive/warehouse/customer

Hadoop/HDFS



HiveQL SELECT

```
SELECT [DISTINCT] <attrList>  
FROM <tableExpr>  
[WHERE <condition>]  
[GROUP BY <attrList>]  
[HAVING <condition>]  
[ORDER BY <attr> [ASC|DESC], ... ]  
[LIMIT <number>]
```

HiveQL Joins

- FULL OUTER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, CROSS JOIN, LEFT SEMI JOIN, JOIN (INNER JOIN)

```
SELECT sales.*, customers.* FROM  
sales JOIN customers  
ON sales.customerid=sales.customerid
```

HiveQL GROUP BY

- Soportado sobre cualquier número de columnas o expresiones.
- Funciones de agregado: MIN, MAX, COUNT, SUM, AVG
- Clausula HAVING aplicada DESPUES del GROUP BY, el resultado es un filtro de grupo agregado

```
SELECT sales.*, customers.* FROM  
sales JOIN customers  
ON sales.customerid=sales.customerid
```

Sub Consultas

- Solo en la clausula FROM

```
SELECT col1, col2
FROM (SELECT col1, col2 ... FROM Table_1 ... ) t
GROUP BY col1, col2
```

Demostración



Hive Queries

Agenda

- Introducción a Hive
- Arquitectura Hive
- HiveQL y Almacenamiento
- UDF y MapReduce
- **Particiones y cubos**
- Navegando Hive con Excel
- Estrategias de Join
- Evolución Hive: Tez y LLAP

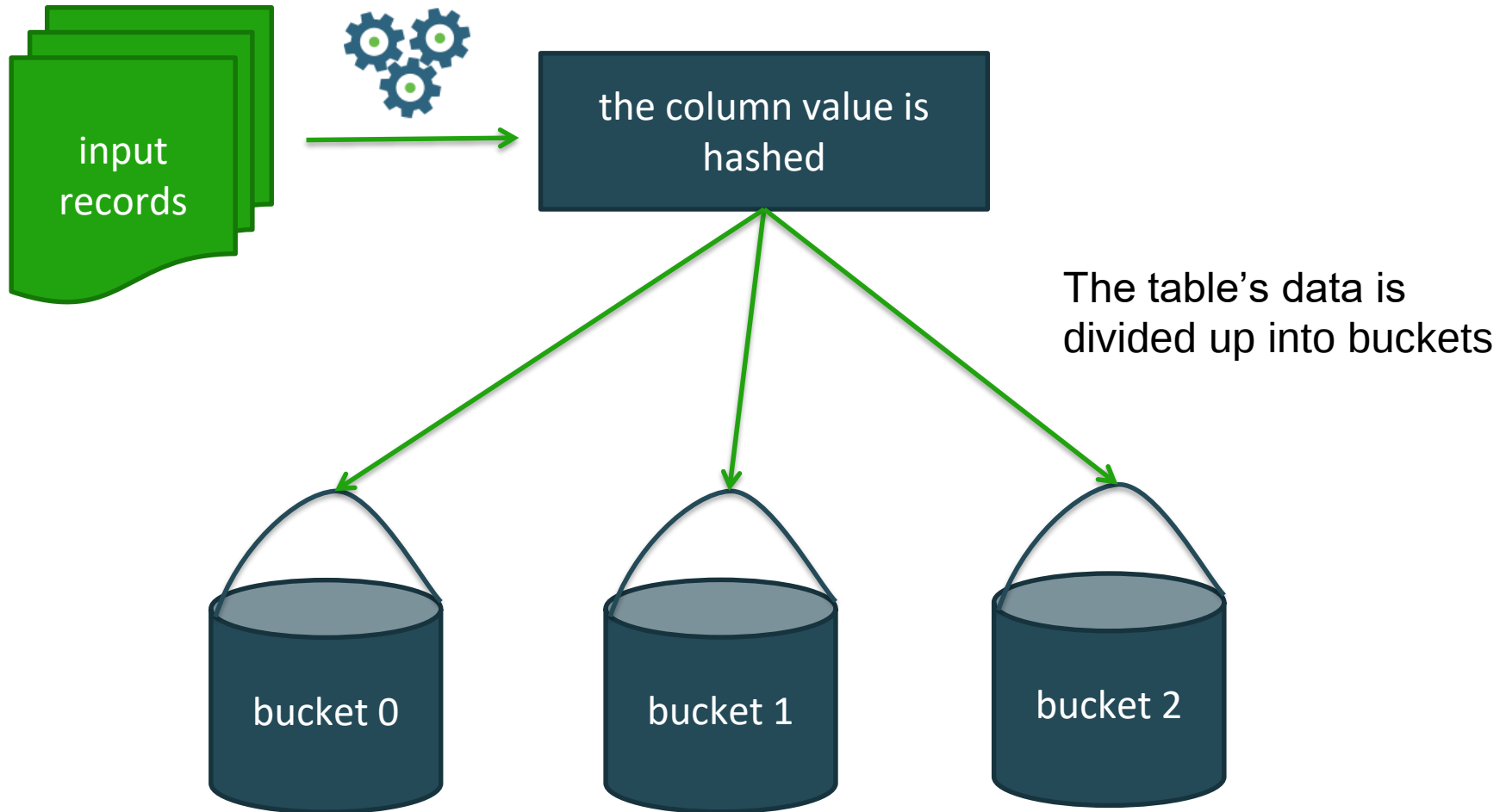
Particiones

- Definidas en tiempo de creación de la tabla utilizando PARTITIONED BY
- Subdirectorios anidados a nivel de Sistema de ficheros

```
CREATE TABLE logs (ts BIGINT, line STRING)  
PARTITIONED BY (dt STRING, country STRING);
```

```
LOAD DATA LOCAL INPATH 'input/hive/partitions/file1'  
INTO TABLE logs  
PARTITION (dt='2001-01-01', country='GB');
```

Hive buckets



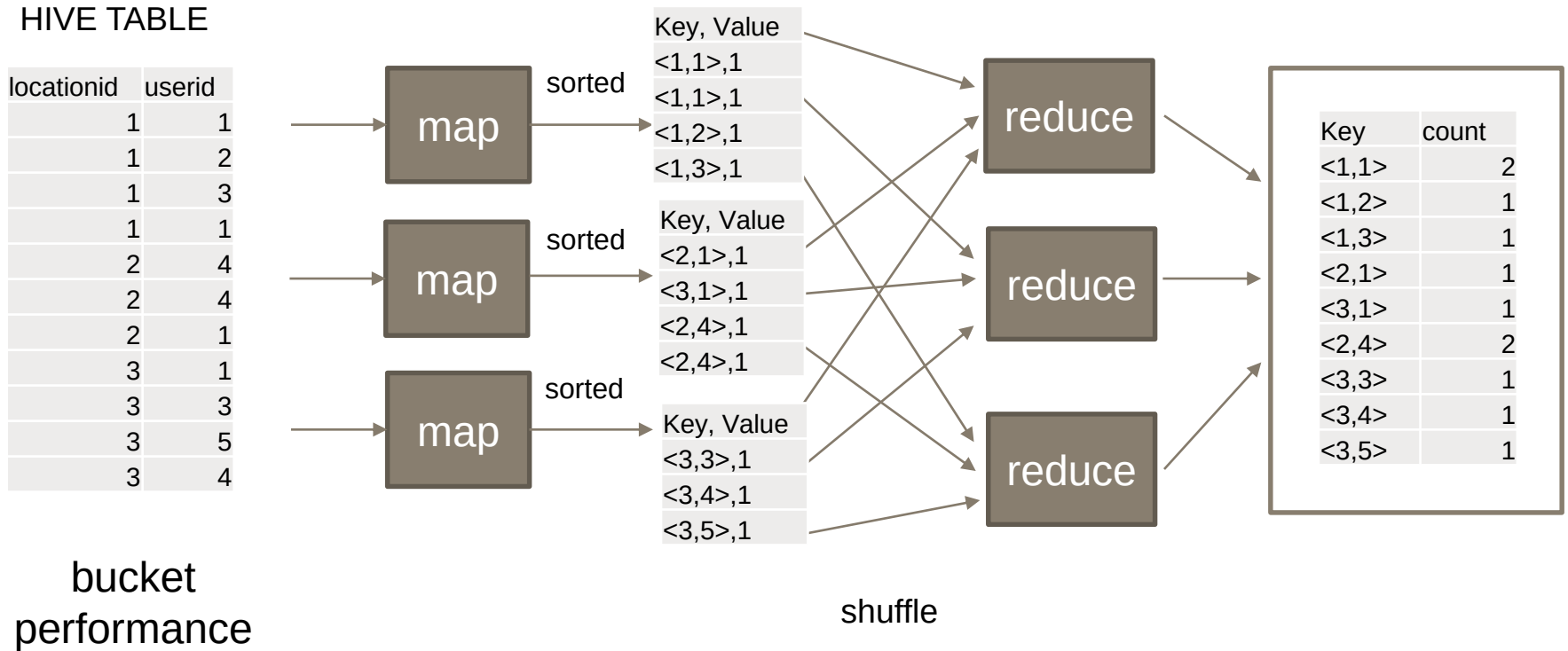
Buckets

- Posibilita consultas más eficientes, porque se ejecuta Map Reduce cuando se consulta
- Ejemplo de rendimiento MAP JOINS

```
CREATE TABLE bucketed users (id INT, name STRING)  
CLUSTERED BY (id) INTO 4 BUCKETS;
```

Group by location, userid

► 3 Mappers, 3 Reducers



Skewed tables

```
CREATE TABLE Customers (  
    id int,  
    username string,  
    zip int  
)  
SKEWED BY (zip) ON (57701, 57702)  
STORED as DIRECTORIES;
```

Distribute By

```
insert overwrite table mytable  
  select gender,age,salary  
  from salaries  
  distribute by age;
```

```
insert overwrite table mytable  
  select gender,age,salary  
  from salaries  
  distribute by age  
  sort by age;
```

Almacenando resultados

INSERT OVERWRITE DIRECTORY

```
' /user/train/ca_or_sd/'  
from names  
  
  select name, state  
  where state = 'CA'  
  or state = 'SD';
```

INSERT OVERWRITE LOCAL DIRECTORY

```
' /tmp/myresults/'  
SELECT * FROM bucketnames  
ORDER BY age;
```

Propiedades Map Reduce

```
SET mapreduce.job.reduces = 12
```

```
hive -f myscript.hive  
-hiveconf mapreduce.job.reduces=12
```

```
SELECT * FROM names  
WHERE age = ${age}  
hive -f myscript.hive -hivevar age=33
```

Demostración



Particiones y Buckets

Agenda

- Introducción a Hive
- Arquitectura Hive
- HiveQL y Almacenamiento
- Particiones y cubos
- **Estrategias de Join**
- Evolución Hive: Tez y LLAP

Estrategias de Join

Tipo	Aproximación	Pros	Cons
Shuffle Join	Las claves de Join se “barajan” usando MapReduce y los joins se hacen en el Reduce	Funciona independientemente del tamaño o estructura de datos	Más recursos consume y más lento
Map (Broadcast) Join	Se cargan las tablas pequeñas en memoria en todos los nodos, mapper escanea a través de la table más grande y hace el join	Muy rápido, un solo escaneo de la tabla más grande	Todas las tablas menos una deben de ser lo suficientemente pequeñas como para caber en RAM
Sort-Merge-Bucket Join	Mappers se benefician de la co-localización de claves para hacer más eficiente el join	Muy rápido para tablas de cualquier tamaño	Datos deben de estar ordenados y clusterizados anteriormente

Shuffle Joins

customer

first	last	id
Nick	Toner	11911
Jessie	Simonds	11912
Kasi	Lamers	11913
Rodger	Clayton	11914
Verona	Hollen	11915

orders

cid	price	quantity
4150	10.50	3
11914	12.25	27
3491	5.99	5
2934	39.99	22
11914	40.50	10

`SELECT * FROM customer JOIN orders ON customer.id = orders.cid;`

{ id: 11911, { first: Nick, last: Toner } }
 { id: 11914, { first: Rodger, last: Clayton } }
 ...



{ id: 11911, { first: Nick, last: Toner } }
 { cid: 4150, { price: 10.50, quantity: 3 } }
 ...

{ cid: 4150, { price: 10.50, quantity: 3 } }
 { cid: 11914, { price: 12.25, quantity: 27 } }
 ...



{ id: 11914, { first: Rodger, last: Clayton } }
 { cid: 11914, { price: 12.25, quantity: 27 } }

Map (Broadcast) Join

customer				orders		
first	last	id		cid	price	quantity
Nick	Toner	11911		4150	10.50	3
Jessie	Simonds	11912		11914	12.25	27
Kasi	Lamers	11913		3491	5.99	5
Rodger	Clayton	11914		2934	39.99	22
Verona	Hollen	11915		11914	40.50	10

`SELECT * FROM customer JOIN orders ON customer.id = orders.cid;`

{ id: 11914, { first: Rodger, last: Clayton } }
{ cid: 11914, { price: 12.25, quantity: 27 },
cid: 11914, { price: 12.25, quantity: 27 } }



Records are joined during
the map phase.

Sort-Merge-Bucket Joins

customer				orders		
first	last	id		cid	price	quantity
Nick	Toner	11911		4150	10.50	3
Jessie	Simonds	11912		11914	12.25	27
Kasi	Lamers	11913		11914	40.50	10
Rodger	Clayton	11914	←	12337	39.99	22
Verona	Hollen	11915		15912	40.50	10

```
SELECT * FROM customer join orders ON customer.id = orders.cid;
```

Distribute and sort by the most common join key.

```
CREATE TABLE orders (cid int, price float, quantity int)  
CLUSTERED BY(cid) SORTED BY(cid) INTO 32 BUCKETS;
```

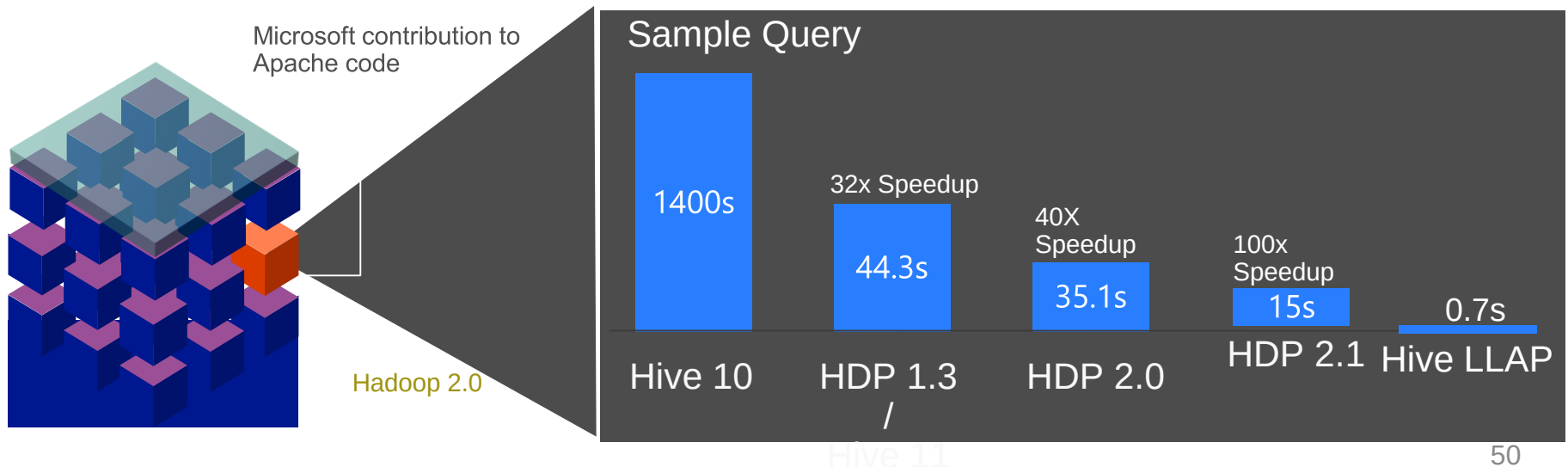
```
CREATE TABLE customer (id int, first string, last string)  
CLUSTERED BY(id) SORTED BY(id) INTO 32 BUCKETS;
```

Agenda

- Introducción a Hive
- Arquitectura Hive
- HiveQL y Almacenamiento
- Particiones y cubos
- Estrategias de Join
- **Evolución Hive: Tez y LLAP**

La iniciativa Stinger / TEZ

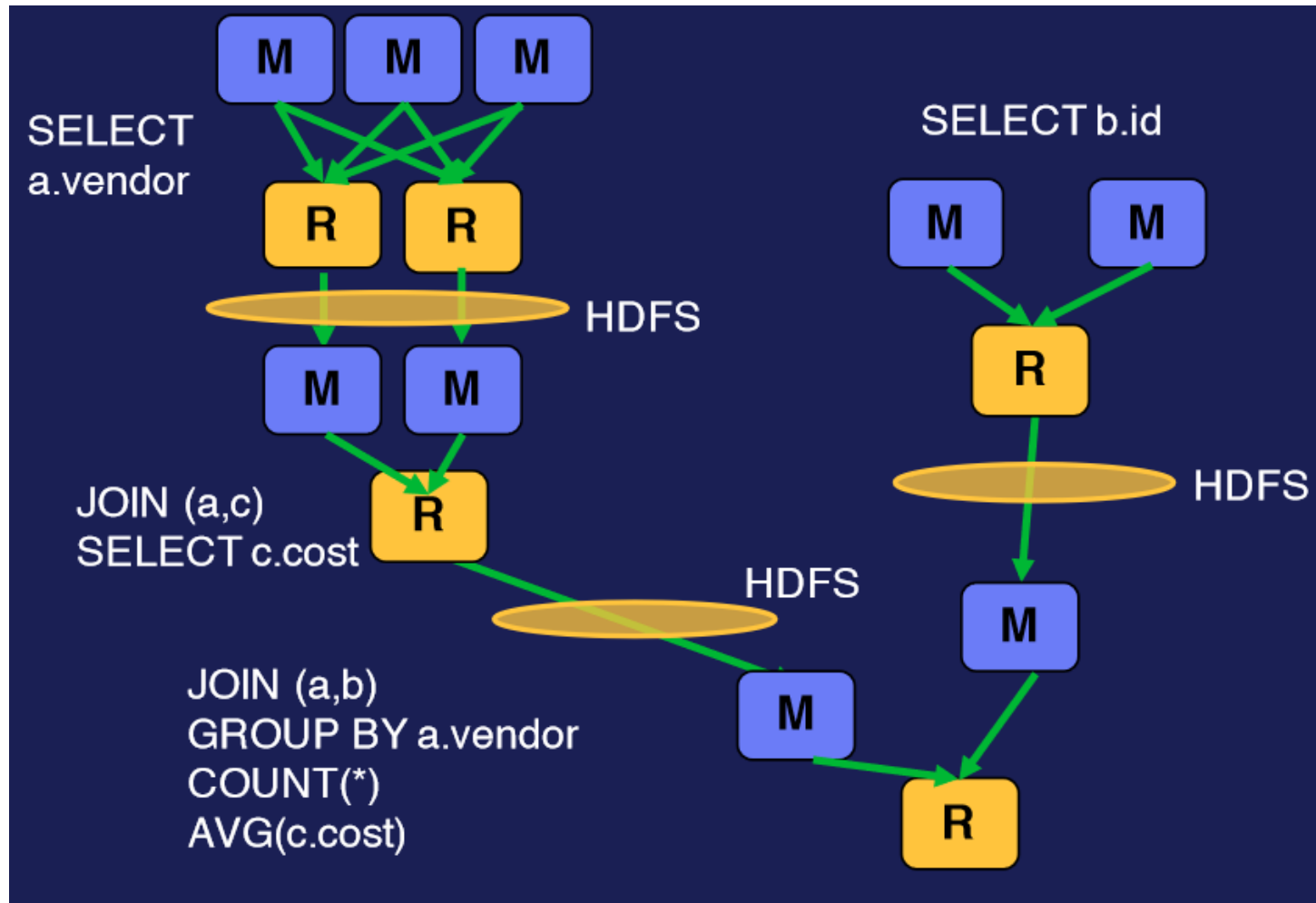
- Construido sobre YARN
- Fast Interactive Query
 - Mejoras de 100x de rendimiento para HIVE
 - Mejora HIVEQL para proporcionar compatibilidad SQL



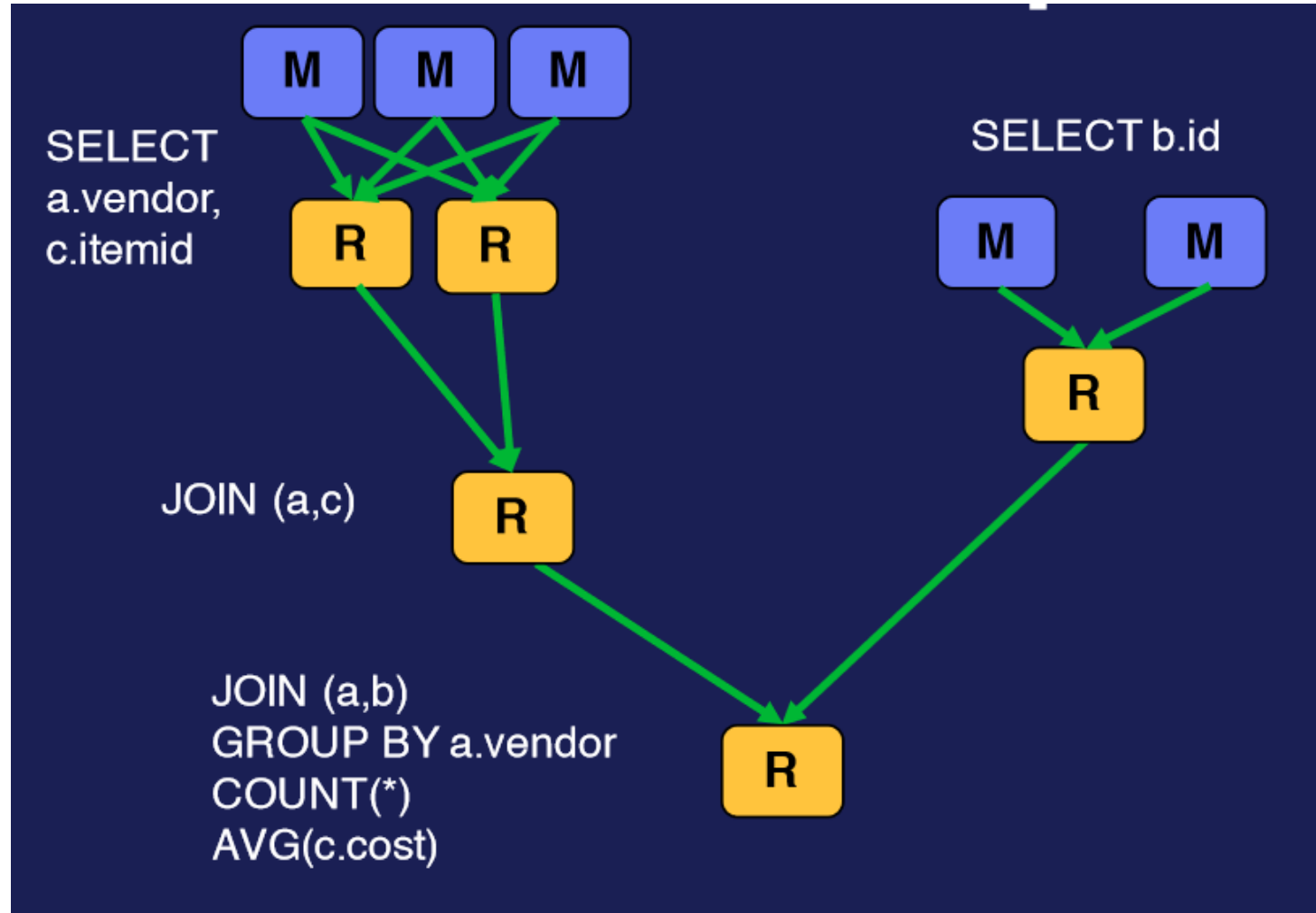
Imagina esta consulta.....

```
SELECT a.vendor, COUNT(*), AVG(c.cost) FROM a  
JOIN b ON (a.id = b.id)  
JOIN c ON (a.itemid = c.itemid)  
GROUP BY a.vendor
```

Con MapReduce....



Con TEZ...





Trabajando con Hive

Vistas Materializadas

- Se calculan los contenidos en creación
- Atómicas
- Utilizadas de forma automática por el optimizador
- Refresco de datos manual
 - Incremental (solo para inserts)

Sintaxis

```
CREATE MATERIALIZED VIEW [IF NOT EXISTS] [db_name.]materialized_view_name
[DISABLE REWRITE]
[COMMENT materialized_view_comment]
[PARTITIONED ON (col_name, ...)]
[
  [ROW FORMAT row_format]
  [STORED AS file_format]
  | STORED BY 'storage.handler.class.name' [WITH SERDEPROPERTIES (...)]
]
[LOCATION hdfs_path]
[TBLPROPERTIES (property_name=property_value, ...)]
AS
<query>;
```

```
ALTER MATERIALIZED VIEW [db_name.]materialized_view_name REBUILD;
```

